

Felhők hálózati szolgáltatásai laboratórium
(BMEVITMMB11)

Vorleser
OCR alkalmazás

Kovács Bertalan
RHDBLM

2026. május 10.



Tartalomjegyzék

1. CI/CD környezet	– 3 –
1.1. Projekt struktúra	– 3 –
1.2. A verziókezelő rendszer struktúrájának elve	– 3 –
1.3. A Flask alkalmazás bemutatása	– 4 –
1.4. Az OKD erőforrások létrehozása	– 5 –
1.5. Folyamatos integráció	– 6 –
1.6. Összegzés	– 8 –
2. Weboldal + OCR detektálás funkció	– 10 –
2.1. Projekt struktúra	– 10 –
2.2. Komponensek rövid bemutatása és technológia választások indoklása	– 11 –
2.3. A komponensek kommunikációjának áttekintése	– 11 –
2.4. A fejlesztett komponensek felületes vizsgálata	– 13 –
2.4.1. Maulwurf	– 13 –
2.4.2. Leserate	– 13 –
2.4.3. Papagei	– 13 –
2.5. A CI/CD folyamat áttekintése	– 14 –
2.6. Az RBAC hiba elhárítása	– 14 –
2.7. A működés bemutatása	– 15 –
3. Értesítési alrendszer	– 16 –
3.1. Architektúrális változások	– 16 –
3.2. Adatbázis-kezelés és robusztus feliratkozás	– 16 –
3.3. Üzenetsor perzisztencia	– 18 –
3.4. Telegram integráció és biztonságos konfigurációkezelés	– 18 –
3.5. Vezérlőpult, API proxy és kliens-oldali hibakezelés	– 18 –
Hivatkozások	– 19 –

1. CI/CD környezet

Ez a fejezet a Vorleser programhoz készült CI/CD környezet konfigurálását és beállítását taglalja egy egyszerű, két komponensből álló, [Python](#) alapú, [Flask](#) szolgáltatáson keresztül. A CI/CD rendszer a [Git](#) alapú [GitHub](#) szolgáltatást használja verziókezelő és integrációs, az [OKD](#)-t pedig telepítési és futtatási környezetnek. OKD implementációnak a [BME által futtatott](#) OKD rendszert választottam.

? Miért OKD?

Az OKD a natív Kubernetes fölé építve olyan fejlesztői (PaaS) funkciókat biztosít, mint az integrált CI/CD – amelynek köszönhetően a telepítési folyamatok könnyen triggerelhetők GitHubról, és a rendszer a forráskódból automatikusan le is buildeli a szükséges Docker image-eket –, valamint a beépített szoftveres útválasztás és a robusztus belső DNS alapú szolgáltatás-felfedezés. Ezek a funkciók kellenek igazán a lazán csatolt mikroszolgáltatások kommunikációjához.

1.1. Projekt struktúra

A példában egy kétszintű Flask alkalmazást hoztam létre. A projekt könyvtárszerkezete a következő:

```
vorleser/  
|-- backend/  
|   |-- app.py  
|   |-- requirements.txt  
|   |-- Dockerfile  
|-- frontend/  
|   |-- app.py  
|   |-- requirements.txt  
|   |-- Dockerfile  
|-- .github/workflows/OKD_deploy.yaml
```

A projekt megtalálható a GitHubon: <https://github.com/berci9ke101/vorleser>, ez a repozitórium fogja a későbbi Vorleser OCR program verziókezelő rendszerét biztosítani, tehát ide fognak majd a későbbi fájlok is kerülni.

1.2. A verziókezelő rendszer struktúrájának elve

A mikroszolgáltatások elveit követve minden komponens saját könyvtárral és Dockerfile-al rendelkezik, minimalizálva a függőségeket és így elősegítve a könnyű hordozhatóságot. A példában az alábbi Dockerfile-t használtam mindkét komponensnél (backend és frontend):

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

A későbbi OCR alkalmazás struktúrája is ezeken az alapokon fog működni.

1.3. A Flask alkalmazás bemutatása

A CI/CD pipeline bemutatásához egy egyszerű Flask alkalmazást hoztam létre, ami-
ben a backend publikál egy szövegrészletet a `/api/data` útvonalán. Ezt a szöveget
felhasználva a frontend megjeleníti a backend üzenetét. A forráskódok alább olvashatóak:

Backend:

```
from flask import Flask, jsonify

app = Flask(__name__)
@app.route('/api/data')
def get_data():
    return jsonify(message="Hello World!")

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Frontend:

```
from flask import Flask
import requests
import os

app = Flask(__name__)
@app.route('/')
def index():
    try:
        response = requests.get(f"http://{os.getenv('BACKEND_URL')}
        ↪ :5000/api/data")
        data = response.json()
        return f"<h1>Backend says: {data['message']}</h1>"
    except Exception as e:
        return f"<h1>Error: {str(e)}</h1>"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=3000)
```

A frontend a `BACKEND_URL` környezeti változón meghatározott néven keresztül éri el

a backendet. Ezt később be is állítottam.

1.4. Az OKD erőforrások létrehozása

Miután bejelentkeztem a BME webes felületén az OKD konzolba, kimásoltam a bejelentkezési parancsomat, majd az OKD platformon az alábbi parancsokkal inicializáltam a projektet és a szolgáltatásokat:

```
# Bejelentkezés az OKD parancssori alkalmazásába
oc login --token=<..> \
    --server=https://api.okd.fured.cloud.bme.hu:6443

# A projekt létrehozása
oc new-project vorleser

# A build-ek létrehozása GitHub forrásból, ügyelve a kontextusra
oc new-app https://github.com/berci9ke101/vorleser.git \
    --context-dir=/backend \
    --name=vorleser-backend \
    --strategy=docker

oc new-app https://github.com/berci9ke101/vorleser.git \
    --context-dir=/frontend \
    --name=vorleser-frontend \
    --strategy=docker

# Szolgáltatások helyezése a komponensek elé és a portjaik kinyitása
oc expose deployment/vorleser-backend --port=5000 --target-port=5000
oc expose deployment/vorleser-frontend --port=3000 --target-port=3000

# A frontend szolgáltatás kiengedése alapértelmezett tanúsítvánnyal
oc expose svc/vorleser-frontend
oc patch route vorleser-frontend \
    -p '{"spec":{"tls":{"termination":"edge"}}}'
```

Az OKD lehetővé teszi, hogy a verziókezelő rendszerben az egyes szolgáltatáskomponensek külön könyvtárban helyezkedjenek el. Ekkor az egyes komponensek kontextusát, a verziókezelő rendszerben elhelyezkedő, elérési útvonaluk alapján be kell állítani a fentebb látott `-context-dir=<..>` kapcsoló segítségével.

Az alkalmazás elérhetőségéhez szolgáltatásokat hoztam létre a komponensek elé és kiengedtem a frontendet a nyílt Internetre. Mivel a klaszter alapértelmezetten a HTTPS használatára kényszerít, így az alapértelmezett bejövő tanúsítványt kellett használnom, azaz be kellett állítani az *Edge TLS termination*-t [1].

A frontendnek megadtam a fentebb már említett backend elérhetőségét környezeti változóként:

```
oc set env deployment/vorleser-frontend BACKEND_URL=vorleser-backend
```

1.5. Folyamatos integráció

A folyamatos integrációt a GitHub Actions segítségével valósítottam meg. Minden `main` ágra történő push esemény automatikusan elindítja a kód szintaktikai ellenőrzését. Ahhoz, hogy az OKD értesüljön az új verzióról a GitHub action bejelentkezik a BME OKD szerverére és értesíti az új verzióról. Ezután az OKD elkezd a Docker image-ek buildelését.

Az autentikációhoz létre kell hozni egy `serviceaccount`-ot. Ennek a menetét az alábbi parancsokkal végeztem el¹:

```
cat <<EOF | oc apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: custom-build-instantiator
  namespace: vorleser
rules:
- apiGroups: ["build.openshift.io"]
  resources: ["buildconfigs/instantiate"]
  verbs: ["create"]
EOF

oc create rolebinding github-trigger-strict-binding \
  --role=custom-build-instantiator \
  --serviceaccount=vorleser:github-trigger \
  -n vorleser

oc create sa github-trigger
oc create token github-trigger --duration=8760h
```

A sikerességről a következő képek számolnak be:

```
berci9ke101@BINARY-DOLPHIN:~$ cat <<EOF | oc apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: custom-build-instantiator
  namespace: vorleser
rules:
- apiGroups: ["build.openshift.io"]
  resources: ["buildconfigs/instantiate"]
  verbs: ["create"]
EOF

oc create rolebinding github-trigger-strict-binding \
  --role=custom-build-instantiator \
  --serviceaccount=vorleser:github-trigger \
  -n vorleser
role.rbac.authorization.k8s.io/custom-build-instantiator created
rolebinding.rbac.authorization.k8s.io/github-trigger-strict-binding created
berci9ke101@BINARY-DOLPHIN:~$ |
```

1. ábra. A megfelelő role létrehozása.

¹A szükséges parancsok a BME OKD üzemeltetője, Bodor András útmutatása alapján születtek.

```

berci9ke101@BINARY-DOLPHIN:/mnt/c/Users/Bertalan/Nextcloud/Egyetem/_MSc/3/LABOR/vorleser$ oc create sa github-trigger
oc create token github-trigger --duration=8760h
serviceaccount/github-trigger created
eyJhbGciOiJSUzI1NiIsImtpZCI6IHRwbG9kRmRmNWpyUWwLLWdRcXVudT3NEWEN4Q1U1Zkg4M2c0YmQ1dmdkSFUifQ.eyJhdwQiOi0lsiaHR0cHM6Ly9rdWw3

```

2. ábra. A token létrehozása a rolehoz.

A kapott tokenet el kell tárolni a GitHub repozitóriumban az **Action secrets** fül alatt:

Actions secrets / New secret

Name *

OKD_TOKEN

Secret *

QiOnsibmFtZSI6ImdpdGh1Yi10cmInZ2VyliwidWlkjoiNTViZGJhZDktMjNjY500YjZjLTIIMTQtODkzYjcwNjQ3NmE1In19L

Add secret

3. ábra. A GitHub Actions token eltárolása.

Maga az action az alábbi yaml fájl segítségével fut le [2]:

```

name: Trigger OKD Build

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.9'

      - name: Install dependencies and Pylint
        run: |
          python -m pip install --upgrade pip
          pip install pylint
          pip install -r backend/requirements.txt
          pip install -r frontend/requirements.txt

      - name: Run Pylint
        run: |

```

```
pylint backend/app.py frontend/app.py

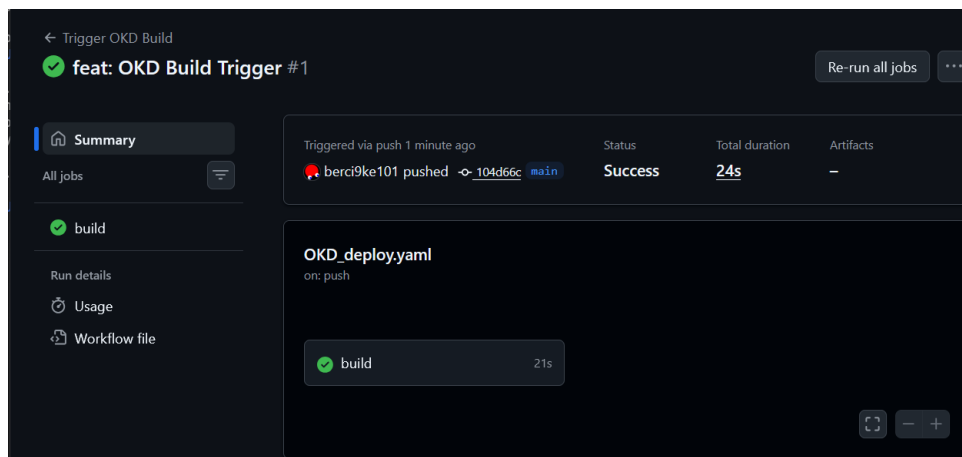
- name: Install oc CLI
  uses: redhat-actions/openshift-tools-installer@v1
  with:
    oc: 4

- name: Login to OKD
  uses: redhat-actions/oc-login@v1
  with:
    openshift_server_url:
      ↪ https://api.okd.fured.cloud.bme.hu:6443
    openshift_token: ${ secrets.OKD_TOKEN }
    namespace: vorleser

- name: Start Backend Build
  run: oc start-build vorleser-backend

- name: Start Frontend Build
  run: oc start-build vorleser-frontend
```

Az action sikeres lefutása után már nézhetjük is az OKD webes felületén a, ahogy a komponenseink elindulnak:

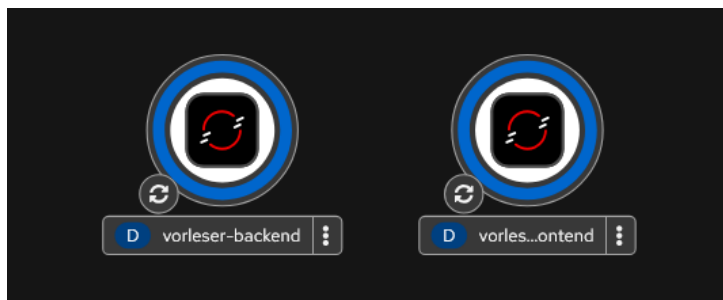


4. ábra. Sikeres GitHub Actions lefutás.

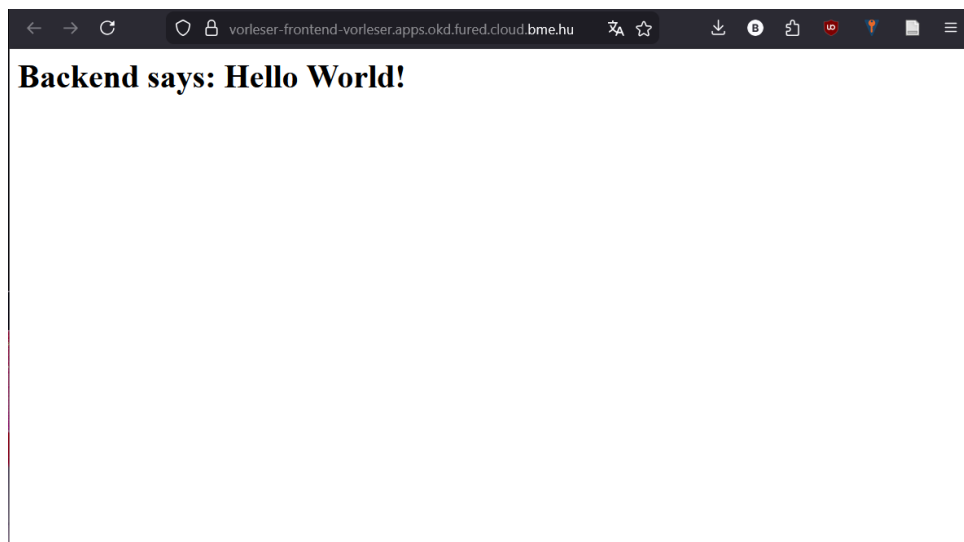
Megtekintve a <https://vorleser-frontend-vorleser.apps.okd.fured.cloud.bme.hu/> linket láthatjuk, hogy működik az alkalmazás:

1.6. Összegzés

A jelen fejezetben bemutatott folyamat elsősorban a Vorleser alkalmazás alapjául szolgáló CI/CD környezet és az OKD platform képességeinek demonstrálására fókuszált. A felépített architektúra egy leegyszerűsített, kétkomponensű Flask alkalmazáson keresztül igazolta a GitHub Actions és az OKD integrációjának zökkenőmentes működését.



5. ábra. Folyamatban lévő build-ek az OKD konzolon.



6. ábra. A működő alkalmazás.

Fontos megjegyezni, hogy ez az implementáció csupán egy kezdeti, „*proof of concept*” fázis volt az alapvető telepítési folyamatok hitelesítésére. A végleges optikai karakterfelismerő (OCR) mikroszolgáltatás-rendszer bevezetésekor ez a bemutatott alap egy jóval kiforrottabb, „kipolírozott” CI/CD pipeline-ná fog bővülni. A végleges folyamat a kódminőség szigorúbb ellenőrzését, komplexebb tesztelési fázisokat és a mikroszolgáltatások egymástól független, finomhangolt frissítését is automatizálni fogja a stabil felhős üzemeltetés érdekében.

2. Weboldal + OCR detektálás funkció

Ez a fejezet a Vorleser alkalmazás elosztott architektúráját és a fejlesztés során hozott technológiai döntéseket mutatja be. A rendszer egy mikroszolgáltatás-alapú ökoszisztémára épül, ahol az egyes funkciókat – mint a webes interfész (*Papagei*), az OCR feldolgozás (*Leseratte*) és a bináris adattárolás (*Maulwurf*) – önálló, saját fejlesztésű, egymástól lazán csatolt konténerizált komponensek látják el. A fejezet részletezi az aszinkron feladatkezelést biztosító *RabbitMQ* (*Briefstaube*) üzenetsor integrációját, a *EasyOCR* könyvtár alkalmazását, valamint a *PostgreSQL* (*Elefant*) alapú perzisztens adattárolást.

2.1. Projekt struktúra

A Vorleser alkalmazás jelenlegi struktúrája a következő (dokumentáció, licenszelés stb. elhagyásával):

```
vorleser/
|-- .github/workflows/OKD_deploy.yaml # CI/CD pipeline leíró
|-- k8s/ # Deklaratív kubernetes leírók
|   |-- blob-storage-deployment.yaml
|   |-- blob-storage-pvc.yaml
|   |-- frontend-deployment.yaml
|   |-- ocr-worker-deployment.yaml
|   |-- postgres-deployment.yaml
|   |-- postgres-pvc.yaml
|   |-- rabbitmq-deployment.yaml
|-- leseratte/ # OCR Worker
|   |-- app.py
|   |-- requirements.txt
|   |-- Dockerfile
|-- maulwurf/ # Blob Storage
|   |-- app.py
|   |-- requirements.txt
|   |-- Dockerfile
|   |-- test_app.py
|-- papagei/ # Frontend & API Gateway
|   |-- templates/
|   |   |-- 404.html
|   |   |-- index.html
|   |   |-- view.html
|   |-- app.py
|   |-- requirements.txt
|   |-- Dockerfile
|-- docker-compose.yaml # Lokális teszteléshez
```

A projekt megtalálható a GitHubon: <https://github.com/berci9ke101/vorleser>

2.2. Komponensek rövid bemutatása és technológia választások indoklása

- **Papagei (Frontend & API Gateway):** A *Papagei* nem csupán a webes alkalmazás, hanem egy átjáró, amely összefogja a *Maulwurf* (tároló), az *Elefant* (adatbázis) és a *Brieftaube* (üzenetsor) hívásait. Konténerizáltam, hogy az OKD-ban egyszerűen publikálható és elérhető legyen kívülről.
- **Leseratte (OCR feldolgozó):** Az OCR művelet (EasyOCR modellek betöltése és futtatása) lassú és számításigényes. Ha ez a frontendben futna, a felhasználói felület megállna. Ha megnő a feltöltött képek száma, példányszáma növelhető anélkül, hogy a frontendhez hozzá kellene nyúlni.

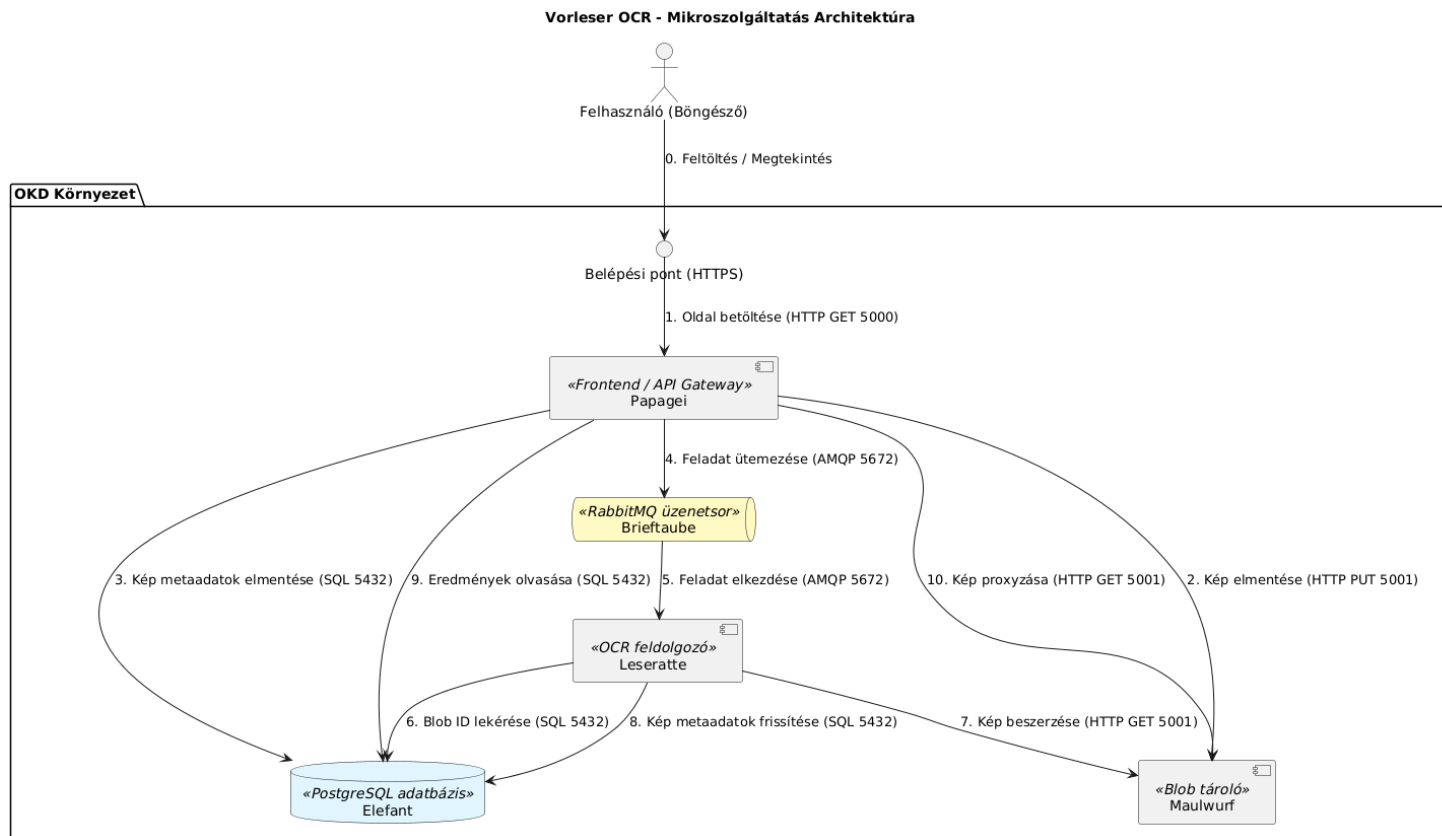
? *Miért nem FaaS?*

Bár az OCR feladat elméletileg alkalmas lenne FaaS (Function as a Service) megoldásra a párhuzamos végrehajtás miatt, a konténeres megoldást választottam, mert az EasyOCR modellek több gigabájtosak és betöltésük jelentős időt vesz igénybe, ami egy FaaS filozófiába nem illene bele.

- **Maulwurf (Blob tároló):** Ahelyett, hogy minden pod közvetlenül csatlakozna egy megosztott fájlrendszerhez, a *Maulwurf* egy egységes API-t biztosít a szolgáltatásoknak a képek tárolására. Ez lehetővé teszi, hogy később a fájlrendszert akár S3-ra vagy más felhős tárolóra cseréljük. Egy PVC biztosítja a fájlok integritását egy újraindítást követően.
- **Elefant (PostgreSQL adatbázis):** A képek metaadatait az `images` táblában (`id`, `blob_id`, `description`, `status`, `detected_text`) relációs formában tároljuk a konzisztencia miatt. A `detected_text` oszlop `JSONB` típusa lehetővé teszi az OCR által talált szövegek és koordináták rugalmas, mégis strukturált tárolását. Egy PVC biztosítja az adatok integritását pod újraindítást követően is.
- **Brieftaube (RabbitMQ üzenetsor):** A *Papagei* (Frontend & API Gateway) és a *Leseratte* (OCR feldolgozó) közötti laza csatolást az `ocr_tasks` nevű üzenetsor valósítja meg. Ez garantálja a feladatok megőrzését a feldolgozó egység túlterheltsége vagy elérhetetlensége esetén is, hiszen a feldolgozás közben megszakadt munkafolyamatok üzenetei automatikusan visszakerülnek a sorba az ismételt végrehajtáshoz.

2.3. A komponensek kommunikációjának áttekintése

A rendszer belső adatforgalmát a mikroszolgáltatások közötti laza csatolás és a hibatűrő üzenetküldés jellemzi. Az [alábbi](#) ábra az ökoszisztéma belső interakcióit és a folyamatok logikai sorrendjét mutatja be a kép feltöltésétől az eredmények megjelenítéséig.



7. ábra. A Vorleser architektúrájának áttekintése.

A folyamat menete (ld. 7. ábra): a felhasználó feltölti a képet (0.), amit a *Papagei* fogad (1.), elment a *Maulwurf* tárolóba (2.) és rögzít az *Elefant* adatbázisban (3.). Ezután aszinkron feladatüzenetet küld a *Brieftaube* üzenetsorba (4.) a feldolgozás megkezdéséhez.

A *Leseratte* worker kiveszi az üzenetet (5.), megszerzi a Blob ID-t (6.) és letölti a képet a tárolóból (7.), majd az EasyOCR motorral elvégzi a szövegfelismerést. Az eredményeket és a sikeres állapotot (8.) közvetlenül az *Elefant* adatbázisba jegyzi vissza.

Végül a felhasználó a *Papagei* felületén keresztül megtekinti az eredményeket (9.). Ez az elosztott felépítés garantálja, hogy a feladatok a komponensek esetleges leállása esetén sem vesznek el.

2.4. A fejlesztett komponensek felületes vizsgálata

2.4.1. Maulwurf

Több meglévő Blob tároló implementációt is használhattam volna, de az egyik egyetemi tárgy keretén belül csináltam már egy kezdetleges Blob tárolót, amit kiegészítettem a házi feladathoz.

Maga a szolgáltatás pofon egyszerű, egyetlen végpontot publikál, melyen a `PUT`, `GET` és `DELETE` *HTTP* metódusokkal lehet műveleteket, adott azonosítójú fájlokon, végezni. Ezen azonosítók célszerűen *UUID*k, mivel felhasználok a *Papagei* lekérdezéseinél és így óhatatlanul kikerülnek az *URL* mezőbe.

A bináris tároló teszteléséhez a Flask keretrendszer beépített tesztelési támogatását használtam [3]. A tesztelési folyamat során a `tempfile` modul segítségével ideiglenes könyvtárakat hoztam létre, így biztosítva, hogy a tesztek izolált környezetben fussanak, és ne módosítsák a tényleges perzisztens tárolót [4]. Ezek a tesztek a CI/CD folyamat során is lefutnak.

2.4.2. Leseratte

A *Leseratte* aszinkron worker a *Brieftaube* feladatait dolgozza fel. Az OCR-modellek a Docker build során kerülnek a konténerbe a hálózati függetlenség érdekében [5]. A 8080-as porton futó `/health` végpont jelzi a készenléte (200) vagy az inicializálást (503). A folyamat során a worker a *Maulwurf*-tól kért képet elemzi, az eredményt pedig az *Elefant* adatbázisba menti.

Az OCR feldolgozó egység az adatbázis-műveletekhez a `psycopg2` könyvtárat használja, amely hatékony interfészt biztosít a PostgreSQL eléréséhez [6]. A komponens Dockerfile-ja az EasyOCR hivatalos ajánlásai alapján készült, különös tekintettel a szükséges rendszerfüggőségekre (pl. `libgl1-mesa-dev`) [7]. A konténerizált környezetben a `PYTHONUNBUFFERED=1` környezeti változó beállítása biztosítja, hogy az alkalmazás naplói azonnal megjelenjenek a standard kimeneten, megkönnyítve a hibakeresést az OKD konzolján [8].

2.4.3. Papagei

A *Papagei* a Vorleser alkalmazás központi belépési pontja és API átjárója, amely a felhasználói felület kiszolgálásáért és a mikroszolgáltatások közötti koordinációért felel. A szolgáltatás kezeli a képek feltöltését, továbbítja a bináris adatokat a *Maulwurf* tárolóba, rögzíti a kezdeti metaadatokat az *Elefant* adatbázisban, majd aszinkron feladatot ütemez a *Brieftaube* üzenetsorba. Emellett egy belső proxy-t is biztosít, amely lehetővé teszi a tárolt képek megjelenítését².

A frontend a Flask sablonkezelő rendszerét alkalmazza a dinamikus HTML tartalom előállításához [9]. A felismert szövegek vizuális megjelenítésekor a rendszer abszolút pozícionált `div` elemeket hoz létre réteggént az eredeti kép felett. Ez a megoldás nem csak a találatok bekeretezését teszi lehetővé, hanem a felhasználó közvetlenül ki is jelölheti és másolhatja a képen látható szöveget.

²A proxy-ra a *CORS* (Cross-Origin Resource Sharing) korlátozások elkerülése miatt van szükség, mivel a bináris adatok egy szeparált belső mikroszolgáltatáson helyezkednek el.

2.5. A CI/CD folyamat áttekintése

A Vorleser projekt folyamatos integrációját egy összetett GitHub Actions munkafolyamat automatizálja, amely minden `main` ágra történő feltöltéskor (*push*) aktiválódik. A folyamat két fő szakaszra tagolódik: ellenőrzés és tesztelés, valamint az OKD alapú telepítés.

A munkafolyamat első szakaszában a kódminőség biztosítása érdekében elkészítettem egy **lint** és egy **test** fázist, amely a `pylint` és a `python-unittest` segítségével ellenőrzi az implementált három mikroszolgáltatás szintaktikai és stilisztikai megfelelőségét, valamint elvégzi a tesztelésüket.

A második szakaszban indul el a **deploy** fázis, amely az ellenőrzött kódból elkészíti a Docker képfájlokat, létrehozza a megfelelő erőforrásokat az OKD környezetben (`PVC`, `service`, `route`, `deployment`), valamint elindítja a megfelelő konténereket.

Mivel a Kubernetes alapértelmezetten nem tartalmaz a Docker Compose `depends_on` paraméteréhez hasonló függőségkezelést, a szolgáltatások indulási sorrendjét *initContainer*-ekkel szabályozom [10]. Ezek a segédkonténerek hálózati ellenőrzésekkel (ebben a projektben leginkább `nc -z` parancsokkal) várakoznak az adatbázis és az üzenetsor elérhetőségére, mielőtt a fő alkalmazás elindulna.

2.6. Az RBAC hiba elhárítása

A telepítési folyamat során jogosultsági problémák merültek fel az OKD klaszteren, mivel a CI/CD triggert végző fióknak nem volt elegendő joga a `PVC`-k kezeléséhez. A hiba elhárításához a `github-trigger` service account-hoz hozzárendeltem az `edit` szerepkört a projekt névterében. A módosítás után a `oc describe rolebinding edit` parancs igazolta, hogy a jogosultságok megfelelően frissültek.

```
# Az edit role hozzárendelése a service account-hoz
oc adm policy add-role-to-user edit \
system:serviceaccount:vorleser:github-trigger -n vorleser

# A beállítás ellenőrzése
oc get rolebindings -n vorleser

Name:          edit
Role:
  Kind: ClusterRole
  Name:  edit
Subjects:
  Kind      Name      Namespace
  ----      -
ServiceAccount github-trigger vorleser
```

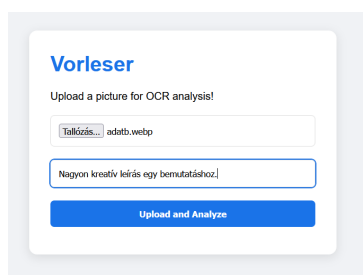
2.7. A működés bemutatása

Ahhoz, hogy elérjem az újonnan elkészített szolgáltatás lekértem a publikus címét az OKD-től:

```
# Külső route elkérése
oc get route vorleser-papagei

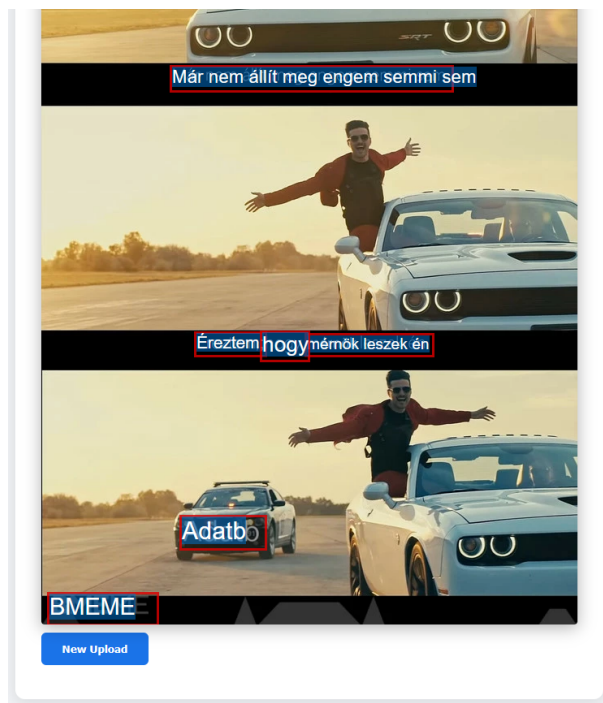
NAME                                HOST/PORT
vorleser-pap...    vorleser-papagei-vorleser.apps.okd.fured.cloud.bme.hu
```

Ezek után meglátogattam a <https://vorleser-papagei-vorleser.apps.okd.fured.cloud.bme.hu/> oldalt és feltöltöttem egy képet, illetve adtam leírást a képnek (ld. 8. ábra). Ezek után megtekintettem az analizált képet (ld. 9. ábra).



The screenshot shows the 'Vorleser' web application interface. At the top, it says 'Upload a picture for OCR analysis!'. Below this is a file upload button labeled 'Töltsd fel...' with the file name 'adatb.webp'. Underneath is a text input field containing the prompt 'Nagyon kreatív leírás egy bemutatáshoz!'. At the bottom is a blue button labeled 'Upload and Analyze'.

8. ábra. A feltöltés menete.



9. ábra. A kész kép oldala.

A beolvasott kép bármikor megtekinthető a https://vorleser-papagei-vorleser.apps.okd.fured.cloud.bme.hu/view/c93b3a4e-0c7e-4210-bb41-5652c5dc6eb9_adatb.webp oldalon.

3. Értesítési alrendszer

A projekt továbbfejlesztése során a rendszer kiegészült egy aszinkron értesítési alrendszerrel, amelynek központi eleme a **Klammeraffe (Értesítési komponens)** nevű mikroszolgáltatás. Ennek a modulnak a feladata, hogy a sikeresen feldolgozott OCR feladatokról valós időben Telegram értesítést küldjön a feliratkozott felhasználóknak, valamint egy belső API-n keresztül kiszolgálja az üzemeltetői vezérlőpult mindenkori adatait az adatbázis alapján.

3.1. Architektúrális változások

Az értesítési funkció bevezetésével az architektúra jelentősen kibővült. A *Papagei* kiegészült egy dedikált vezérlőpult felülettel, amelynek hálózati kéréseit belső proxy-n keresztül továbbítja a *Klammeraffe* szolgáltatás felé. Az új architektúra felépítését a 10. ábra szemlélteti.

A munkafolyamat a következőképpen bővült:

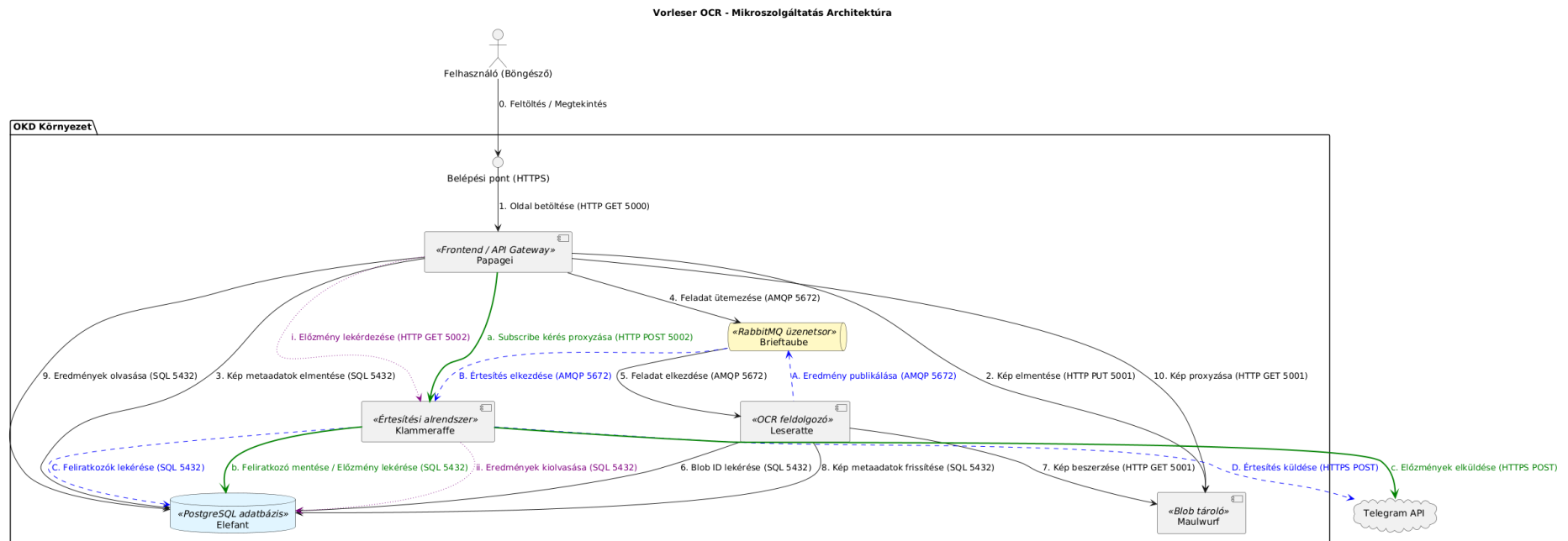
- **Értesítési adatfolyam (Kék útvonal):** miután a *Leseratte* sikeresen elvégzi a karakterfelismerést és frissíti a PostgreSQL adatbázist, egy új üzenetet helyez el a *Brieftaube* `ocr_notifications` nevű sorában. A *Klammeraffe* ezt a sort figyeli, kiolvassa az eredményeket, lekérdezi az aktív feliratkozók listáját az *Elefant* adatbázisból, végül *HTTP* kéréseken keresztül kommunikál a *Telegram API*-val.
- **Feliratkozási adatfolyam (Zöld útvonal):** Amikor egy operátor a weben keresztül regisztrálja a Telegram azonosítóját, a *Papagei* a kérést belső proxy-ként továbbítja a *Klammeraffe* felé (*HTTP POST*). A szolgáltatás rögzíti az új feliratkozót az *Elefant* adatbázisban, majd azonnal lekérdezi az eddigi OCR elemzések előzményeit. Végül ezeket az előzményeket egy kezdő szinkronizációs üzenet formájában továbbítja az új felhasználónak a *Telegram API*-n keresztül.
- **Vezérlőpult frissítési adatfolyam (Lila útvonal):** A webes vezérlőpult táblázatának feltöltéséhez a *Papagei* egy *HTTP GET* kéréssel fordul a *Klammeraffe* belső `/api/history` végpontjához. A *Klammeraffe* ekkor kiolvassa az *Elefant* adatbázisból a korábbi eredményeket, amelyeket a frontend dinamikusan, kliens oldalon dolgoz fel és jelenít meg.

3.2. Adatbázis-kezelés és robusztus feliratkozás

A *Klammeraffe* az állapotmegőrzéshez egy új táblát (`subscriptions`) inicializál az *Elefant* adatbázisban, amely a felhasználók egyedi *Telegram Chat ID*-jeit tárolja. Mivel az elosztott rendszerekben előfordulhat, hogy egy felhasználó többször is megpróbál feliratkozni (például a hálózat lassúsága vagy újratöltés miatt), a konzisztencia megőrzés jegyében az SQL beszúrás egy úgynevezett *UPSERT* logikát használ:

```
INSERT INTO subscriptions (chat_id)
VALUES (%s) ON CONFLICT (chat_id) DO NOTHING;
```

Ez a megoldás garantálja az adatbázis konzisztenciáját és megelőzi a duplikált azonosítók miatti SQL hibákat [11].



10. ábra. A Vorleser kiterjesztett mikroszolgáltatás architektúrája.

Az új folyamatok menete (ld. 10. ábra): Az **értesítési adatfolyam** (szaggatott kék vonalak) során, miután a *Leseratte* befejezi az OCR feldolgozást, az eredményt publikálja a *Brieftaube* üzenetsorba (A.). A *Klammeraffe* ezt az üzenetet fogadva megkezd az értesítési folyamatot (B.), lekéri az aktív feliratkozókat az *Elefant* adatbázisból (C.), majd egy *HTTP POST* kérés segítségével elküldi számukra a valós idejű riasztást a *Telegram API*-n keresztül (D.).

A **feliratkozási adatfolyam** (vastag zöld vonalak) egy új operátor regisztrációjakor indul. A *Papagei* a webes felületről érkező feliratkozási kérést proxy-ként továbbítja a *Klammeraffe* felé (a.). A szolgáltatás elmenti az új feliratkozót és azonnal lekéri az eddigi elemzések előzményeit az *Elefant* adatbázisból (b.), végül egy kezdő szinkronizációs üzenet formájában továbbítja azokat az új felhasználó *Telegram* fiókjára (c.).

Végül a **vezérlőpult frissítési adatfolyama** (pontozott lila vonalak) a vezérlőpult kiszolgálásáért felel. A *Papagei* egy *HTTP GET* kéréssel lekérdezi az előzményeket a *Klammeraffe* belső végpontjáról (i.), amely az *Elefant* adatbázisból olvassa ki a korábbi eredményeket (ii.), így biztosítva a táblázat valós idejű és dinamikus tartalmát.

3.3. Üzenetsor perzisztencia

Az elosztott aszinkron rendszerek esetében kritikus szempont, hogy egyetlen esemény se vesszen el a komponensek esetleges újraindulása (például egy OKD rollout vagy skálázás) során. Az új komponens fejlesztése során a *Briefstaube* konfigurációját a hibatűrő működéshez igazítottam:

1. **Durable Queues:** Az `ocr_notifications` sor tartós (`durable=True`) beállítással jön létre, így maga az üzenetsor túléli a RabbitMQ szerver újraindulását [12].
2. **Persistent Messages:** A *Leseratte* az elkészült feladatokat `delivery_mode=2` (perzisztens) tulajdonsággal publikálja [12].

Ez a megközelítés garantálja, hogy ha a *Klammeraffe* éppen nem elérhető, az elküldendő értesítések biztonságban várakoznak a sorban, amíg az új pod fel nem pörög.

3.4. Telegram integráció és biztonságos konfigurációkezelés

A Telegram üzenetküldés *HTTP POST* kéréseken keresztül valósul meg a hivatalos *Telegram Bot API* `sendMessage` végpontján [13]. A JSON adatszerkezetből a nyers OCR felismert szövegeket a Python kód összefűzi, méretüket a Telegram platform korlátaival igazítja, mielőtt beillesztené az üzenetbe. A vizuálisan tagolt megjelenés érdekében a `parse_mode="HTML"` paramétert használtam [14].

A *Telegram Bot API* tokenje szenzitív adat, melynek a `Deployment` fájlokban való pusztta tárolása komoly biztonsági kockázatot jelentett volna. Ennek kivédésére az OKD klaszteren egy natív `Secret` erőforrást (`telegram-token`) hoztam létre. A *Klammeraffe* Deployment leírója az `env.valueFrom.secretKeyRef` direktíva segítségével futásidőben olvassa ki és injektálja ezt a token `TELEGRAM_TOKEN` környezeti változóként a konténerbe, garantálva, hogy a kód verziókezelőjébe ne kerüljön fel érzékeny adat.

3.5. Vezérlőpult, API proxy és kliens-oldali hibakezelés

A *Papagei* szolgáltatás kiegészült egy új üzemeltetői felülettel (`/subscribe`). Mivel az architektúrában a mikroszolgáltatások szigorúan szeparáltak, a frontend dedikált proxy végpontokon keresztül továbbítja a `/api/history` és `/api/subscribe` lekérdezéseket a *Klammeraffe* felé, áthidalva ezzel a hálózati elérési korlátokat.

Kliens oldalon a vezérlőpult táblázatának dinamikus generálása során egy egyszerű biztonsági mechanizmust vezettem be. Abban az esetben, ha egy képből nem nyerhető ki szöveg (például érvénytelen formátum, PDF feltöltése vagy feldolgozási hiba miatt), a PostgreSQL adatbázisban a szövegmező `null` értéket vesz fel. A többi mező nem jelent problémát, hiszen az adatbázisséma meggátolja a `null` érték felvételét.

```
const fullText = Array.isArray(row.text)
? row.text.map(t => t.text).join(' ')
: (row.text || "");
```

Hivatkozások

- [1] OKD Community, *Creating advanced routes - Creating a route using the default certificate through an Ingress object*. Elérhető: https://docs.okd.io/4.16/networking/routes/creating-advanced-routes.html#nw-ingress-edge-route-default-certificate_creating-advanced-routes (Letöltve: 2026. 03. 15.).
- [2] Red Hat, *OpenShift Login GitHub Action*. Elérhető: <https://github.com/redhat-actions/oc-login> (Letöltve: 2026. 03. 15.).
- [3] Pallets Projects, *Testing Flask Applications*. Elérhető: <https://flask.palletsprojects.com/en/stable/testing> (Letöltve: 2026. 04. 19.).
- [4] Pallets Projects, *Test Coverage*. Elérhető: <https://flask.palletsprojects.com/en/stable/tutorial/tests> (Letöltve: 2026. 04. 19.).
- [5] JaiedAI, *EasyOCR*. Elérhető: <https://github.com/JaiedAI/EasyOCR#usage> (Letöltve: 2026. 04. 19.).
- [6] The Psycopg Team, *Basic module usage*. Elérhető: <https://www.psycopg.org/docs/usage.html> (Letöltve: 2026. 04. 19.).
- [7] A. Challis, *EasyOCR/Dockerfile*. Elérhető: <https://github.com/JaiedAI/EasyOCR/blob/master/Dockerfile> (Letöltve: 2026. 04. 19.).
- [8] Zeitounator, *django - What is the use of PYTHONUNBUFFERED in docker file?* Elérhető: <https://stackoverflow.com/questions/59812009/what-is-the-use-of-pythonunbuffered-in-docker-file/59812588#59812588> (Letöltve: 2026. 04. 19.).
- [9] GeeksforGeeks, *Flask Rendering Templates*. Elérhető: <https://www.geeksforgeeks.org/python/flask-rendering-templates> (Letöltve: 2026. 04. 19.).
- [10] D. Wilkin, *Kubernetes Deployment Dependencies*. Elérhető: <https://medium.com/google-cloud/kubernetes-deployment-dependencies-ef703e563956> (Letöltve: 2026. 04. 19.).
- [11] PostgreSQL Global Development Group, *PostgreSQL: Documentation: 18: INSERT*. Elérhető: <https://www.postgresql.org/docs/current/sql-insert.html> (Letöltve: 2026. 05. 10.).
- [12] Broadcom, *RabbitMQ tutorial - Work Queues - Message Durability*. Elérhető: <https://www.rabbitmq.com/tutorials/tutorial-two-python#message-durability> (Letöltve: 2026. 05. 10.).
- [13] Telegram, *Telegram Bot API - sendMessage*. Elérhető: <https://core.telegram.org/bots/api#sendMessage> (Letöltve: 2026. 05. 10.).
- [14] Telegram, *Telegram Bot API - HTML style*. Elérhető: <https://core.telegram.org/bots/api#html-style> (Letöltve: 2026. 05. 10.).